

TRIGGERS

TRIGGERS

A database trigger is a stored PL/SQL program unit associated with a specific database table.

ORACLE executes (fires) a database trigger automatically when a given SQL operation (like INSERT, UPDATE or DELETE) affects the table.

- Unlike a procedure, or a function, which must be invoked explicitly, database triggers are invoked implicitly.

TRIGGERS

Database triggers can be used to perform any of the following:

- Audit data modification
- Log events transparently
- Enforce complex business rules
 - Derive column values automatically
- Implement complex security authorizations
 - Maintain replicate tables

TRIGGERS

- You can associate up to 12 database triggers with a given table. A database trigger has three parts: a **triggering event**, an **optional trigger constraint**, and a **trigger action**.
- When an event occurs, a database trigger is fired, and a predefined PL/SQL block will perform the necessary action.

TRIGGERS

SYNTAX:

```
CREATE [OR REPLACE] TRIGGER  
    trigger_name  
{BEFORE|AFTER} triggering_event ON  
    table_name  
    [FOR EACH ROW]  
    [WHEN condition]  
    DECLARE  
    Declaration statements  
    BEGIN  
    Executable statements  
    EXCEPTION  
    Exception-handling statements  
    END;
```

TRIGGERS

The trigger_name references the name of the trigger.

BEFORE or AFTER specify when the trigger is fired (before or after the triggering event).

The triggering_event references a DML statement issued against the table (e.g., INSERT, DELETE, UPDATE).

The table_name is the name of the table associated with the trigger.

The clause, FOR EACH ROW, specifies a trigger is a row trigger and fires once for each modified row.

A WHEN clause specifies the condition for a trigger to be fired.

Bear in mind that if you drop a table, all the associated triggers for the table

TYPES OF TRIGGERS

Triggers may be called BEFORE or AFTER the following events:

INSERT, UPDATE and DELETE.

The before/after options can be used to specify when the trigger body should be fired with respect to the triggering statement.

If the user indicates a BEFORE option, then Oracle fires the trigger before executing the triggering statement. On the other hand, if an AFTER is used, Oracle fires the trigger after executing the triggering statement.

TYPES OF TRIGGERS

- A trigger may be a ROW or STATEMENT type.
- If the statement FOR EACH ROW is present in the CREATE TRIGGER clause of a trigger, the trigger is a row trigger. A row trigger is fired for each row affected by a triggering statement.
- A statement trigger, however, is fired only once for the triggering statement, regardless of the number of rows affected by the triggering statement.

TYPES OF TRIGGERS

Example: statement trigger

```
CREATE OR REPLACE TRIGGER mytrig1 BEFORE DELETE OR
INSERT OR UPDATE ON employee
BEGIN
    IF (TO_CHAR(SYSDATE, 'day') IN ('sat', 'sun')) OR
    (TO_CHAR(SYSDATE, 'hh:mi') NOT BETWEEN '08:30' AND
    '18:30') THEN
        RAISE_APPLICATION_ERROR(-20500,
        'table is secured');
    END IF;
    END;
/
```

The above example shows a trigger that limits the DML actions to the employee table to weekdays from 8.30am to 6.30pm.

If a user tries to insert/update/delete a row in the EMPLOYEE table, a warning message will be prompted.

SUMMARY

A trigger in MySQL is a set of SQL statements that reside in a system catalog. **It is a special type of stored procedure that is invoked automatically in response to an event.** Each trigger is associated with a table, which is activated on any DML statement such as **INSERT, UPDATE, or DELETE.**

- A trigger is called a special procedure because it cannot be called directly like a stored procedure. The main difference between the trigger and procedure is that a trigger is called automatically when a data modification event is made against a table. In contrast, a stored procedure must be called explicitly.
- Generally, **triggers are of two types** according to the [SQL](#) standard: row-level triggers and statement-level triggers.

Row-Level Trigger: It is a trigger, which is activated for each row by a triggering statement such as insert, update, or delete. For example, if a table has inserted, updated, or deleted multiple rows, the row trigger is fired automatically for each row affected by the [insert](#), [update](#), or [delete statement](#).

Statement-Level Trigger: It is a trigger, which is fired once for each event that occurs on a table regardless of how many rows are inserted, updated, or deleted.

Why we need/use triggers in MySQL?

We need/use triggers in MySQL due to the following features:

- Triggers help us to enforce business rules.
- Triggers help us to validate data even before they are inserted or updated.
- Triggers help us to keep a log of records like maintaining audit trails in tables.
- SQL triggers provide an alternative way to check the integrity of data.
- Triggers provide an alternative way to run the scheduled task.
- Triggers increases the performance of SQL queries because it does not need to compile each time the query is executed.
- Triggers reduce the client-side code that saves time and effort.
- Triggers help us to scale our application across different platforms.
- Triggers are easy to maintain.

Limitations of Using Triggers in MySQL

- MySQL triggers do not allow to use of all validations; they only provide extended validations. **For example**, we can use the NOT NULL, UNIQUE, CHECK and FOREIGN KEY constraints for simple validations.
- Triggers are invoked and executed invisibly from the client application. Therefore, it isn't easy to troubleshoot what happens in the database layer.
- Triggers may increase the overhead of the database server.

When we use a statement that does not use INSERT, UPDATE or DELETE query to change the data in a table, the triggers associated with the trigger will not be invoked.

Types of Triggers in MySQL?

Here we can define the six types of actions or events in the form of triggers:

Before Insert: It is activated before the insertion of data into the table.

After Insert: It is activated after the insertion of data into the table.

Before Update: It is activated before the update of data in the table.

After Update: It is activated after the update of the data in the table.

Before Delete: It is activated before the data is removed from the table.

After Delete: It is activated after the deletion of data from the table.

example

Suppose you have a employees table and you want to insert a record into an audit_log table automatically Whenever a new employee is added.

1. Table Structures

```
CREATE TABLE employees ( emp_id INT PRIMARY  
KEY, name VARCHAR(100), department  
VARCHAR(50) )  
; CREATE TABLE audit_log ( log_id PRIMARY KEY,  
action_time TIMESTAMP DEFAULT  
CURRENT_TIMESTAMP  
action_description TEXT );
```

trigger

```
CREATE TRIGGER after_employee_insert
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (action_description)
    VALUES (CONCAT('New employee added: ',
NEW.name, ', Department: ',
NEW.department));
END;
```

trigger name: after_employee_insert

Triggering event: AFTER INSERT on the employees table

Action: Inserts a log into audit log describing the new employee

THANK YOU